DATA STRUCTURES AND ALGORITHMS

PROGRAMS RECORD

Name	T. Chandra Lasya
Registration Number	11259a504
Section	S9

Experiment 1 — Unit I

Implementation of Basic Array Operations (Insertion, Deletion, Traversal)

AIM

To implement insertion, deletion and traversal operations on an array.

ALGORITHM

1. Start.
2. Read the array size and elements.
3. Read position and value, shift elements right and insert the value.
4. Read deletion position, shift remaining elements left and reduce size.
5. Traverse the array and display all elements.
6. Stop.

PROGRAM

```
#include <stdio.h>
int main() {
    int a[100], n, i, pos, value, del;
    printf("Enter array size: ");
    scanf("%d", &n);
    printf("Enter array elements: ");
    for(i=0;i<n;i++) scanf("%d",&a[i]);

    printf("Enter position and value to insert: ");
    scanf("%d%d",&pos,&value);
    for(i=n;i>=pos;i--) a[i]=a[i-1];
    a[pos-1]=value; n++;
    printf("After Insertion: ");
    for(i=0;i<n;i++) printf("%d ",a[i]);

    printf("\nEnter position to delete: ");
    scanf("%d",&del);
    for(i=del-1;i<n-1;i++) a[i]=a[i+1];
    n--;
    printf("After Deletion: ");
    for(i=0;i<n;i++) printf("%d ",a[i]);

    printf("\nArray Elements: ");
    for(i=0;i<n;i++) printf("%d ",a[i]);
    return 0;
}
```

RESULT

The program for **Implementation of Basic Array Operations (Insertion, Deletion, Traversal)** was executed successfully and the expected output was obtained.

Experiment 2 — Unit I

Building a Dynamic Employee Record System using Structure Pointers

AIM

To create and display employee records dynamically using structures and pointers.

ALGORITHM

1. Start.
2. Read the number of employees.
3. Allocate memory dynamically for the employee structures.
4. Read employee ID, name and salary using a structure pointer.
5. Display all employee records.
6. Free the allocated memory and stop.

PROGRAM

```
#include <stdio.h>
#include <stdlib.h>

struct Employee {
    int id;
    char name[30];
    float salary;
};

int main() {
    int n, i;
    struct Employee *e;
    printf("Enter number of employees: ");
    scanf("%d",&n);
    e = (struct Employee*)malloc(n * sizeof(struct Employee));

    for(i=0;i<n;i++) {
        printf("Enter ID, Name and Salary: ");
        scanf("%d %s %f",&e[i].id,e[i].name,&e[i].salary);
    }

    printf("\nEmployee Records\n");
    for(i=0;i<n;i++)
        printf("ID: %d  Name: %s  Salary: %.2f\n",
               e[i].id,e[i].name,e[i].salary);

    free(e);
    return 0;
}
```

RESULT

The program for **Building a Dynamic Employee Record System using Structure Pointers** was executed successfully and the expected output was obtained.

Experiment 3 — Unit II

Implementation of Stack and Expression Conversion - Infix to Postfix

AIM

To implement a stack and convert an infix expression into postfix form.

ALGORITHM

1. Start.
2. Read the infix expression.
3. Scan each symbol from left to right.
4. Push '(' onto the stack; for an operand, add it to output.
5. For an operator, pop higher/equal precedence operators and then push it.
6. On ')', pop until '(' is found. Pop remaining operators.
7. Display the postfix expression and stop.

PROGRAM

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>

char stack[100]; int top=-1;
int prec(char c){ return c=='+'||c=='-'?1:c=='*'||c=='/'?2:c=='^'?3:0; }
void push(char c){ stack[++top]=c; }
char pop(){ return stack[top--]; }

int main(){
    char in[100], post[100]; int i,j=0; char c;
    printf("Enter infix expression: ");
    scanf("%s",in);
    for(i=0;in[i];i++){
        c=in[i];
        if(isalnum(c)) post[j++]=c;
        else if(c=='(') push(c);
        else if(c==')'){
            while(top!=-1 && stack[top]!='(') post[j++]=pop();
            pop();
        } else {
            while(top!=-1 && stack[top]!='(' && prec(stack[top])>=prec(c))
                post[j++]=pop();
            push(c);
        }
    }
    while(top!=-1) post[j++]=pop();
    post[j]='\0';
    printf("Postfix expression: %s",post);
    return 0;
}
```

RESULT

The program for **Implementation of Stack and Expression Conversion - Infix to Postfix** was executed successfully and the expected output was obtained.

Experiment 4 — Unit II

Postfix Expression Evaluation using Stack

AIM

To evaluate a postfix expression using a stack.

ALGORITHM

1. Start.
2. Read the postfix expression.
3. Scan each symbol.
4. If it is an operand, push it onto the stack.
5. If it is an operator, pop two operands, perform the operation and push the result.
6. The final stack value is the result.
7. Stop.

PROGRAM

```
#include <stdio.h>
#include <ctype.h>

int s[100], top=-1;
void push(int x){ s[++top]=x; }
int pop(){ return s[top--]; }

int main(){
    char exp[100], c; int i,a,b,r;
    printf("Enter postfix expression: ");
    scanf("%s",exp);
    for(i=0;exp[i];i++){
        c=exp[i];
        if(isdigit(c)) push(c-'0');
        else {
            b=pop(); a=pop();
            if(c=='+') r=a+b;
            else if(c=='-') r=a-b;
            else if(c=='*') r=a*b;
            else r=a/b;
            push(r);
        }
    }
    printf("Result = %d",pop());
    return 0;
}
```

RESULT

The program for **Postfix Expression Evaluation using Stack** was executed successfully and the expected output was obtained.

Experiment 5 — Unit II

Implementation of Simple Queue using Array

AIM

To implement insertion and deletion operations in a simple queue using an array.

ALGORITHM

1. Start and initialize front = -1 and rear = -1.
2. Insert an element at rear; if the queue is empty, set front = 0.
3. Delete an element from front and increment front.
4. Display the queue elements from front to rear.
5. Stop.

PROGRAM

```
#include <stdio.h>
#define MAX 5
int q[MAX], front=-1, rear=-1;

void enqueue(int x){
    if(rear==MAX-1) printf("Queue Overflow\n");
    else { if(front==-1) front=0; q[++rear]=x; }
}
void dequeue(){
    if(front==-1 || front>rear) printf("Queue Underflow\n");
    else printf("Deleted: %d\n",q[front++]);
}
void display(){
    int i;
    if(front==-1 || front>rear) { printf("Queue is empty"); return; }
    printf("Queue: ");
    for(i=front;i<=rear;i++) printf("%d ",q[i]);
}
int main(){
    enqueue(10); enqueue(20); enqueue(30);
    display(); printf("\n");
    dequeue(); display();
    return 0;
}
```

RESULT

The program for **Implementation of Simple Queue using Array** was executed successfully and the expected output was obtained.

Experiment 6 — Unit II

Implementation of Priority Queue using Unordered Array

AIM

To implement a priority queue using an unordered array.

ALGORITHM

1. Start.
2. Insert each item with its priority at the next free position.
3. To delete, search for the element with the highest priority.
4. Remove that element and shift the remaining elements.
5. Display the queue.
6. Stop.

PROGRAM

```
#include <stdio.h>
#define MAX 5
struct Item { int data, priority; } q[MAX];
int n=0;

void insert(int x,int p){ q[n].data=x; q[n].priority=p; n++; }
void deleteMax(){
    int i,k=0;
    for(i=1;i<n;i++) if(q[i].priority>q[k].priority) k=i;
    printf("Deleted: %d (priority %d)\n",q[k].data,q[k].priority);
    for(i=k;i<n-1;i++) q[i]=q[i+1];
    n--;
}
void display(){
    int i;
    printf("Priority Queue: ");
    for(i=0;i<n;i++) printf("(%d,p%d) ",q[i].data,q[i].priority);
}
int main(){
    insert(10,2); insert(20,5); insert(30,1);
    display(); printf("\n");
    deleteMax(); display();
    return 0;
}
```

RESULT

The program for **Implementation of Priority Queue using Unordered Array** was executed successfully and the expected output was obtained.

Experiment 7 — Unit III

Implementation of Singly Linked List (Insertion, Deletion, and Traversal)

AIM

To implement insertion, deletion and traversal operations in a singly linked list.

ALGORITHM

1. Start with head = NULL.
2. Create a node and insert it at the beginning.
3. Traverse from head and display all nodes.
4. Delete the first node by moving head to the next node.
5. Display the updated list and stop.

PROGRAM

```
#include <stdio.h>
#include <stdlib.h>

struct Node { int data; struct Node *next; };
struct Node *head=NULL;

void insert(int x){
    struct Node *n=malloc(sizeof(struct Node));
    n->data=x; n->next=head; head=n;
}
void deleteFirst(){
    struct Node *t=head;
    if(head){ head=head->next; free(t); }
}
void display(){
    struct Node *t=head;
    printf("List: ");
    while(t){ printf("%d -> ",t->data); t=t->next; }
    printf("NULL");
}
int main(){
    insert(30); insert(20); insert(10);
    display(); printf("\n");
    deleteFirst(); display();
    return 0;
}
```

RESULT

The program for **Implementation of Singly Linked List (Insertion, Deletion, and Traversal)** was executed successfully and the expected output was obtained.

Experiment 8 — Unit III

Implementation of Circular Queue using Linked List

AIM

To implement insertion and deletion in a circular queue using a linked list.

ALGORITHM

1. Start with rear = NULL.
2. For insertion, create a node; if empty, point it to itself, otherwise link it after rear and update rear.
3. For deletion, remove the front node, which is rear->next.
4. Display elements by traversing from rear->next until rear is reached.
5. Stop.

PROGRAM

```
#include <stdio.h>
#include <stdlib.h>

struct Node { int data; struct Node *next; };
struct Node *rear=NULL;

void enqueue(int x){
    struct Node *n=malloc(sizeof(struct Node));
    n->data=x;
    if(!rear){ rear=n; n->next=n; }
    else { n->next=rear->next; rear->next=n; rear=n; }
}
void dequeue(){
    struct Node *f;
    if(!rear){ printf("Queue Underflow\n"); return; }
    f=rear->next;
    if(f==rear) rear=NULL;
    else rear->next=f->next;
    printf("Deleted: %d\n",f->data);
    free(f);
}
void display(){
    struct Node *p;
    if(!rear){ printf("Queue is empty"); return; }
    p=rear->next; printf("Circular Queue: ");
    do { printf("%d ",p->data); p=p->next; } while(p!=rear->next);
}
int main(){
    enqueue(10); enqueue(20); enqueue(30);
    display(); printf("\n");
    dequeue(); display();
    return 0;
}
```

RESULT

The program for **Implementation of Circular Queue using Linked List** was executed successfully and the expected output was obtained.

Experiment 9 — Unit IV

Implementation and Traversal (In-order, Pre-order, Post-order) of a Binary Search Tree

AIM

To create a Binary Search Tree and perform in-order, pre-order and post-order traversals.

ALGORITHM

1. Start with root = NULL.
2. Insert each value according to BST rules: smaller values go left and larger values go right.
3. Perform in-order traversal: Left, Root, Right.
4. Perform pre-order traversal: Root, Left, Right.
5. Perform post-order traversal: Left, Right, Root.
6. Stop.

PROGRAM

```
#include <stdio.h>
#include <stdlib.h>

struct Node { int data; struct Node *left,*right; };

struct Node* insert(struct Node* r,int x){
    if(!r){
        r=malloc(sizeof(struct Node));
        r->data=x; r->left=r->right=NULL; return r;
    }
    if(x<r->data) r->left=insert(r->left,x);
    else r->right=insert(r->right,x);
    return r;
}
void inorder(struct Node*r){ if(r){inorder(r->left);printf("%d ",r->data);inorder(r->right);} }
void preorder(struct Node*r){ if(r){printf("%d ",r->data);preorder(r->left);preorder(r->right);} }
void postorder(struct Node*r){ if(r){postorder(r->left);postorder(r->right);printf("%d ",r->data);} }

int main(){
    int a[]={50,30,70,20,40,60,80},i;
    struct Node *root=NULL;
    for(i=0;i<7;i++) root=insert(root,a[i]);
    printf("In-order: "); inorder(root);
    printf("\nPre-order: "); preorder(root);
    printf("\nPost-order: "); postorder(root);
    return 0;
}
```

RESULT

The program for **Implementation and Traversal (In-order, Pre-order, Post-order) of a Binary Search Tree** was executed successfully and the expected output was obtained.

Experiment 10 — Unit V

Implementation of Graph Search Algorithms (BFS and DFS)

AIM

To implement Breadth First Search and Depth First Search for a graph.

ALGORITHM

1. Represent the graph using an adjacency matrix.
2. For BFS, use a queue and visit adjacent unvisited vertices level by level.
3. For DFS, use recursion and visit an unvisited adjacent vertex before backtracking.
4. Print the traversal order for both methods.
5. Stop.

PROGRAM

```
#include <stdio.h>
#define N 5
int g[N][N]={{0,1,1,0,0},{1,0,0,1,0},{1,0,0,0,1},
            {0,1,0,0,1},{0,0,1,1,0}};
int visited[N];

void bfs(int s){
    int q[N],f=0,r=0,i,v;
    for(i=0;i<N;i++) visited[i]=0;
    q[r++]=s; visited[s]=1;
    while(f<r){
        v=q[f++]; printf("%d ",v);
        for(i=0;i<N;i++) if(g[v][i]&&!visited[i]){
            visited[i]=1; q[r++]=i;
        }
    }
}
void dfs(int v){
    int i;
    visited[v]=1; printf("%d ",v);
    for(i=0;i<N;i++) if(g[v][i]&&!visited[i]) dfs(i);
}
int main(){
    int i;
    printf("BFS: "); bfs(0);
    for(i=0;i<N;i++) visited[i]=0;
    printf("\nDFS: "); dfs(0);
    return 0;
}
```

RESULT

The program for **Implementation of Graph Search Algorithms (BFS and DFS)** was executed successfully and the expected output was obtained.

Experiment 11 — Unit V

Implementation of Bubble, Selection and Heap Sorts

AIM

To implement Bubble Sort, Selection Sort and Heap Sort and display the sorted arrays.

ALGORITHM

1. Read the array.
2. Bubble Sort: repeatedly compare adjacent elements and swap if they are out of order.
3. Selection Sort: select the minimum element and place it at the current position.
4. Heap Sort: build a max heap, swap the root with the last element and heapify the reduced heap.
5. Display the sorted arrays and stop.

PROGRAM

```
#include <stdio.h>
void print(int a[],int n){int i;for(i=0;i<n;i++)printf("%d ",a[i]);}

void bubble(int a[],int n){
    int i,j,t;
    for(i=0;i<n-1;i++) for(j=0;j<n-i-1;j++)
        if(a[j]>a[j+1]){t=a[j];a[j]=a[j+1];a[j+1]=t;}
}
void selection(int a[],int n){
    int i,j,k,t;
    for(i=0;i<n-1;i++){k=i;
        for(j=i+1;j<n;j++) if(a[j]<a[k]) k=j;
        t=a[i];a[i]=a[k];a[k]=t;
    }
}
void heapify(int a[],int n,int i){
    int l=2*i+1,r=2*i+2,m=i,t;
    if(l<n&&a[l]>a[m])m=l;
    if(r<n&&a[r]>a[m])m=r;
    if(m!=i){t=a[i];a[i]=a[m];a[m]=t;heapify(a,n,m);}
}
void heap(int a[],int n){
    int i,t;
    for(i=n/2-1;i>=0;i--)heapify(a,n,i);
    for(i=n-1;i>0;i--){t=a[0];a[0]=a[i];a[i]=t;heapify(a,i,0);}
}
int main(){
    int a[]={5,1,4,2,8},b[]={5,1,4,2,8},c[]={5,1,4,2,8};
    bubble(a,5); selection(b,5); heap(c,5);
    printf("Bubble Sort: ");print(a,5);
    printf("\nSelection Sort: ");print(b,5);
    printf("\nHeap Sort: ");print(c,5);
    return 0;
}
```

RESULT

The program for **Implementation of Bubble, Selection and Heap Sorts** was executed successfully and the expected output was obtained.

Experiment 12 — Unit V

Implementation of Hashing with Collision Resolution using Linear Probing

AIM

To implement hashing with collision resolution using linear probing.

ALGORITHM

1. Initialize all hash table positions to -1.
2. Compute index = key % table size.
3. If the position is occupied, increment the index circularly until an empty position is found.
4. Insert the key at the available position.
5. Display the hash table.
6. Stop.

PROGRAM

```
#include <stdio.h>
#define SIZE 10

int main(){
    int h[SIZE],i,key,index,pos;
    for(i=0;i<SIZE;i++) h[i]=-1;

    int keys[]={23,43,13,27};
    for(i=0;i<4;i++){
        key=keys[i];
        index=key%SIZE;
        pos=index;
        while(h[pos]!=-1) pos=(pos+1)%SIZE;
        h[pos]=key;
    }

    printf("Hash Table\n");
    for(i=0;i<SIZE;i++)
        printf("%d : %d\n",i,h[i]);
    return 0;
}
```

RESULT

The program for **Implementation of Hashing with Collision Resolution using Linear Probing** was executed successfully and the expected output was obtained.